



ASIGNATURA:

AUTOMATIZACIÓN DE INFRAESTRUCTURA DIGITAL I

UNIDAD II

SISTEMAS DE CONTROL DE VERSIONES

DOCENTE:

EDGAR BRAVO HUERTA

ALUMNO:

USVALDO JIMENEZ TORRES

8° "A"

**LICENCIATURA EN INGENIERÍA EN
TECNOLOGÍAS DE LA INFORMACIÓN E
INNOVACIÓN DIGITAL**



¿SISTEMA DE CONTROL DE VERSIONES?

¿Qué es el control de versiones?

El control de versiones, también conocido como control de fuente o control de revisiones, es una práctica importante de desarrollo de software para hacer un seguimiento y gestionar los cambios realizados en el código y otros archivos. Está estrechamente relacionado con la gestión del código fuente.

se hace un seguimiento de cada cambio realizado en el código base. Esto permite a los desarrolladores de software ver todo el historial de quién cambió qué en un momento dado, y retroceder de la versión actual a una versión anterior si es necesario. También crea una fuente única de la verdad.

El control de versiones (o control de fuente o control de revisiones) sirve como una red de seguridad para proteger el código fuente de daños irreparables. Gracias a esto, el equipo de desarrollo tiene la libertad de experimentar sin temor a causar daños o crear conflictos de código.

Ejemplos

GitHub

Te permiten crear una cuenta, subir tus carpetas con proyectos y colaborar con otras personas en internet.

- **GitLab:** Es el rival más fuerte de GitHub. Muchas empresas grandes lo prefieren porque te permite automatizar tareas (como revisar que tus archivos o códigos no tengan errores antes de guardarlos definitivamente).
- **Bitbucket:** Esta plataforma es de la empresa Atlassian. Es súper popular en entornos de trabajo profesionales porque se conecta a la perfección con herramientas para organizar tareas en equipo como Jira y Trello.

2. Alternativas independientes y enfocadas en la privacidad

- **Codeberg:** Es una plataforma sin fines de lucro alojada en Europa. Nació como una opción para los desarrolladores y creadores que no quieren que empresas gigantescas (como Microsoft, que es dueña de GitHub) tengan acceso a sus archivos, garantizando máxima privacidad.
- **Gitea:** No es una página web pública como tal, sino un software gratuito y ultra ligero. Te sirve para que tú mismo crees tu propio "GitHub privado" en una computadora que tengas en casa, ideal para proyectos personales que no quieres que nadie más vea en internet.

3. Herramientas especiales para archivos pesados (3D y Videojuegos)

Plataformas como GitHub están pensadas principalmente para archivos de texto (código), por lo que si intentas subir archivos gigantescos (como modelos 3D, texturas pesadas o animaciones), se pueden volver lentas. Para eso existen estas herramientas:

- **Helix TeamHub (Perforce):** Es la plataforma en la nube estándar en la industria de la animación, el cine y los videojuegos. Permite a diseñadores, artistas 3D y programadores subir archivos que pesan gigabytes o terabytes enteros sin que el sistema falle.
- **Unity Version Control (Antes Plastic SCM):** Una herramienta integrada directamente en los motores de desarrollo que almacena de forma muy eficiente escenas completas, personajes, texturas y mapas.

Githup

Conceptos generales

1. Repositorio (Repository o "Repo")

Es la carpeta virtual de tu proyecto alojada en la nube de GitHub. Contiene todos los archivos (código, imágenes, modelos, etc.) y, lo más importante, guarda **todo el historial de cambios** de ese proyecto desde el día uno. Puede ser público (cualquiera lo ve) o privado.

2. Commit (Confirmación de cambio)

Es como guardar una partida en un videojuego o tomar una foto instantánea de tu proyecto en un momento específico. Cada vez que haces un *commit*, le pones un título explicativo (ej. Historial: commit "Se corrigió el color del fondo") para saber exactamente qué cambiaste en ese momento.

3. Branch (Rama)

Es una línea de trabajo separada. Por defecto, tu proyecto tiene una rama principal (llamada main o master). Si quieres experimentar, agregar una función nueva o hacer un cambio arriesgado sin descomponer lo que ya funciona, creas una **rama secundaria**. Si el experimento sale bien, la unes a la principal; si sale mal, simplemente la borras.

4. Clone (Clonar)

Es la acción de descargar una copia exacta de un repositorio que está en internet (GitHub) a tu computadora local para poder trabajar en él. No es una simple descarga, ya que la carpeta descargada se queda "conectada" con GitHub.

5. Push (Subir / Empujar)

Es enviar los cambios y los *commits* que hiciste localmente en tu computadora hacia el servidor de GitHub en internet. Es la forma de actualizar tu proyecto en la nube para que otros lo vean.

6. Pull (Bajar / Jalar)

Es lo contrario al *push*. Sirve para descargar a tu computadora los cambios más recientes que otras personas hayan subido al repositorio de GitHub. Así te aseguras de tener siempre la versión más nueva.

7. Pull Request (PR / Solicitud de extracción)

Es la forma en que pides permiso para unir tus cambios a la rama principal de un proyecto en equipo. Cuando terminas una tarea en tu rama, creas un *Pull Request* para que tus compañeros o profesores revisen tu trabajo, lo comenten y, si todo está correcto, lo aprueben.

8. Fork (Bifurcar)

Es sacar una copia de un repositorio de otra persona y guardarlo directamente en tu propia cuenta de GitHub. Te permite experimentar o modificar un proyecto ajeno de forma totalmente independiente sin afectar al dueño original.

La vinculación GitHub Desktop y Visual Studio

Primero debemos crear la carpeta donde se vinculará las herramientas que estarán en nuestra computadora de escritorio



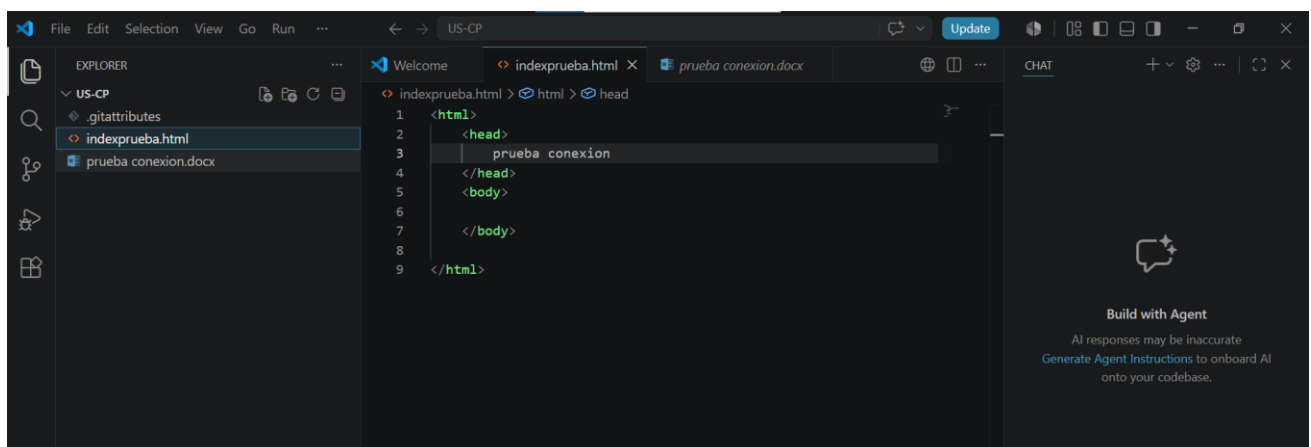
Una vez hecho se instala GitHub Desktop y Visual Studio que vincularemos en la carpeta de escritorio



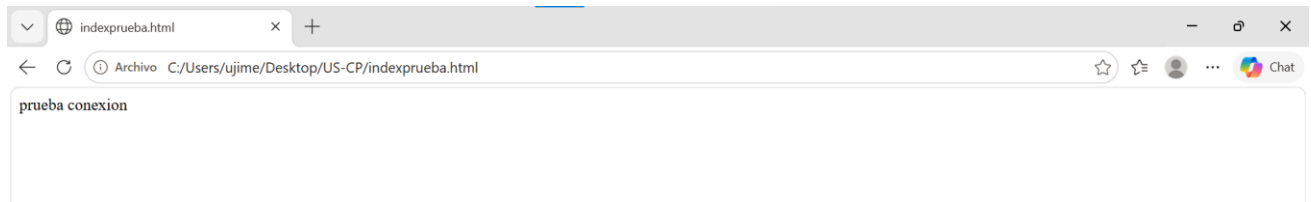
Verificamos que están vinculadas a la carpeta si contiene los archivos que instalamos significa que se está ejecutando de manera correcta.

 .gitattributes	10/06/2026 09:16 a. m.	Archivo de origen Git...	1 KB
 indexprueba	10/06/2026 09:19 a. m.	Microsoft Edge HTM...	1 KB
 prueba conexion	10/06/2026 09:16 a. m.	Documento de Micro...	0 KB

Se verifica si se aguarda en nuestra carpeta empezamos a escribir como título prueba de conexión y verificamos en la carpeta.



Si te aparece esto cuando abres donde lo guardaste y se aparece el título que se le puso eso significa conexión exitosa.



En la aplicación de Git Hub debemos verificar que las carpetas aparezcan.

